Overall detection

1. Content Description

This course implements the acquisition of color images and the use of the MediaPipe framework to detect the entire body, including the face, hands, and torso. This section requires inputting commands in the terminal. The appropriate terminal will be selected based on your motherboard type. This section uses a Raspberry Pi 5 as an example.

For Raspberry Pi and Jetson Nano boards, you need to open a terminal on the host computer and enter the command to enter the Docker container. Once inside the Docker container, enter the commands mentioned in this section in the terminal. For instructions on entering the Docker container from the host computer, refer to **[01. Robot Configuration and Operation Guide] --[5.Enter Docker (For JETSON Nano and RPi 5)]**.

For Orin and RDK motherboards, simply open the terminal and enter the commands mentioned in this section.

2. Program Startup

For the Raspberry Pi 5 and jetson nano controller, you must first enter the Docker container. For the Orin and RDK controller, this is not necessary.

Entering the Docker container (see the Docker course for steps: 4. Docker Startup Script).

All commands below must be executed within the same Docker container.**(See the Docker course for steps: 3. Docker Commit and Multi-Terminal Entry).**

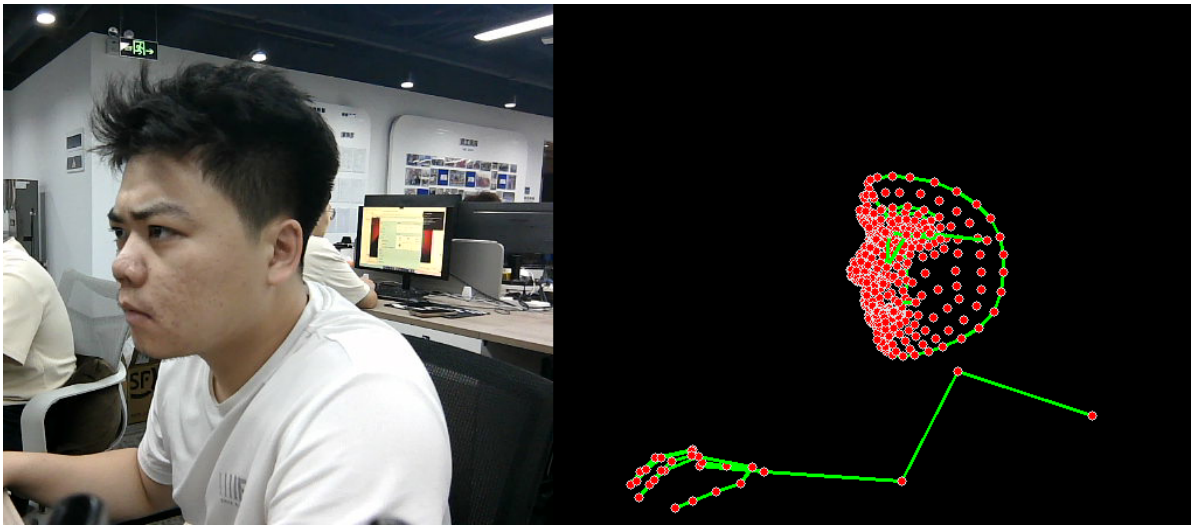
First, in the terminal, enter the following command to start the camera.

```
#usb camera
ros2 launch usb_cam camera.launch.py
#nuwa camera
ros2 launch ascamera hp60c.launch.py
```

After successfully starting the camera, open another terminal and enter the following command to start the overall detection program.

```
ros2 run yahboomcar_mediapipe 03_Holistic
```

After running the program, the image below appears. The detected points are displayed on the right side of the image.



3. Core Code Analysis

Program code path:

- Raspberry Pi 5 and Jetson Nano motherboard

The program code is running in Docker. The path in Docker is `/root/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_mediapipe/yahboomcar_mediapipe/03_Holistic.py`

- Orin motherboard

The program code path is `/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_mediapipe/yahboomcar_mediapipe/03_Holistic.py`

- RDK motherboard

The program code path is `/home/sunrise/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_mediapipe/yahboomcar_mediapipe/03_Holistic.py`

Import the necessary library files.

```
import time
import rclpy
from rclpy.node import Node
import cv2 as cv
import numpy as np
import mediapipe as mp
from geometry_msgs.msg import Point
from yahboomcar_msgs.msg import PointArray
from sensor_msgs.msg import Image
from cv_bridge import CvBridge, CvBridgeError
import os
```

Initialize data and define publishers and subscribers,

```
def __init__(self, staticMode=False, landmarks=True, detectionCon=0.5,
trackingCon=0.5):
    super().__init__('holistic_detector')
    #Use the class in the mediapipe library to define an overall object
```

```
    self.mpHolistic = mp.solutions.holistic
    self.mpFaceMesh = mp.solutions.face_mesh
    self.mpHands = mp.solutions.hands
    self.mpPose = mp.solutions.pose
    self.mpDraw = mp.solutions.drawing_utils
    self.mpholistic = self.mpHolistic.Holistic(
        static_image_mode=staticMode,
        smooth_landmarks=landmarks,
        min_detection_confidence=detectionCon,
        min_tracking_confidence=trackingCon)
    #Define the properties of the joint connection line, which will be used in
the subsequent joint point connection function
    self.lmDrawSpec = mp.solutions.drawing_utils.DrawingSpec(color=(0, 0, 255),
thickness=-1, circle_radius=3)
    self.drawSpec = mp.solutions.drawing_utils.DrawingSpec(color=(0, 255, 0),
thickness=2, circle_radius=2)
    #create a publisher
    self.bridge = CvBridge()
    #Define subscribers for the color image topic
    camera_type = os.getenv('CAMERA_TYPE', 'usb')
    topic_name = '/ascamera_hp60c/camera_publisher/rgb0/image' if camera_type ==
'nuwa' else '/usb_cam/image_raw'
    self.subscription = self.create_subscription(
    Image,
    topic_name,
    self.image_callback,
    10)
```

Color image callback function,

```
def image_callback(self, msg):
    #Use CvBridge to convert color image message data into image data
    frame = self.bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')
    #Pass the obtained image into the defined pubPosePoint function, draw=False
means not to draw the joint points on the original color image
    frame, img = self.findHolistic(frame, draw=True)
    #Merge two images
    combined_frame = self.frame_combine(frame, img)
    cv.imshow('HolisticDetector', combined_frame)
```

findHolistic function,

```
def findHolistic(self, frame, draw=True):
    #Create a new image based on the size of the image passed in. The image data
type is uint8
    img = np.zeros(frame.shape, np.uint8)
    #Convert the color space of the incoming image from BGR to RGB to facilitate
subsequent image processing
    img_RGB = cv.cvtColor(frame, cv.COLOR_BGR2RGB)
    #Call the process function in the mediapipe library to process the image.
During init, the self.mpholistic object is created and initialized.
    self.results = self.mpholistic.process(img_RGB)
    #Determine whether a face is detected. If so, connect each point on the blank
image created previously
    if self.results.face_landmarks:
```

```
        if draw: self.mpDraw.draw_landmarks(frame, self.results.face_landmarks,
self.mpFaceMesh.FACEMESH_CONTOURS, self.lmDrawSpec, self.drawSpec)
        self.mpDraw.draw_landmarks(img, self.results.face_landmarks,
self.mpFaceMesh.FACEMESH_CONTOURS, self.lmDrawSpec, self.drawSpec)
    #Determine if the torso is detected, and if so, connect each point on the
blank image created previously
    if self.results.pose_landmarks:
        if draw: self.mpDraw.draw_landmarks(frame, self.results.pose_landmarks,
self.mpPose.POSE_CONNECTIONS, self.lmDrawSpec, self.drawSpec)
        self.mpDraw.draw_landmarks(img, self.results.pose_landmarks,
self.mpPose.POSE_CONNECTIONS, self.lmDrawSpec, self.drawSpec)
    #Determine whether the left hand is detected. If so, connect each point on
the blank image created previously.
    if self.results.left_hand_landmarks:
        if draw: self.mpDraw.draw_landmarks(frame,
self.results.left_hand_landmarks, self.mpHands.HAND_CONNECTIONS,
self.lmDrawSpec, self.drawSpec)
        self.mpDraw.draw_landmarks(img, self.results.left_hand_landmarks,
self.mpHands.HAND_CONNECTIONS, self.lmDrawSpec, self.drawSpec)
    #Determine whether the right hand is detected. If so, connect each point on
the blank image created previously.
    if self.results.right_hand_landmarks:
        if draw: self.mpDraw.draw_landmarks(frame,
self.results.right_hand_landmarks, self.mpHands.HAND_CONNECTIONS,
self.lmDrawSpec, self.drawSpec)
        self.mpDraw.draw_landmarks(img, self.results.right_hand_landmarks,
self.mpHands.HAND_CONNECTIONS, self.lmDrawSpec, self.drawSpec)
    return frame, img
```

The frame_combine image merging function was mentioned in the first lesson of this chapter. Please refer to [07.Mediapipe Visual Course] - [1. Hand Detection] for an analysis of this function.